



Practica 3
Gabriel Valle Carrasco
Katya Vanessa Torres Robles
17/03/2025

Objetivo

Comprender y repasar el uso de apuntadores realizando las siguientes funciones.

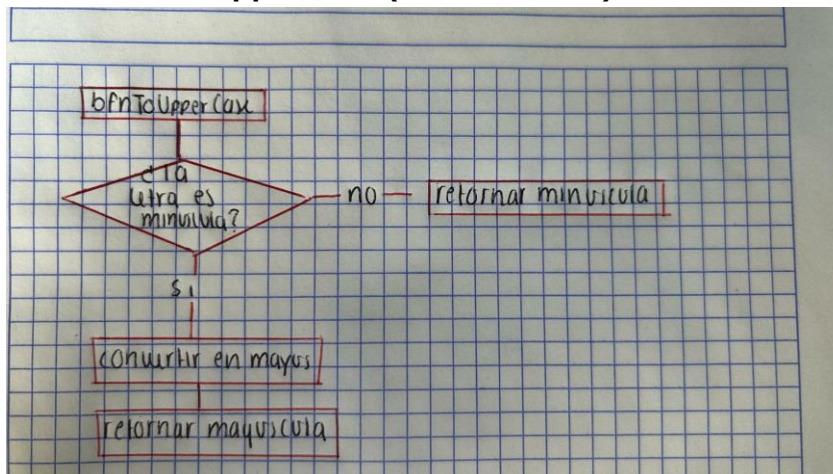
Práctica

Realizar las siguientes funciones en C utilizando el IDE MCUXpresso. Por esta ocasion, sera posible usar las funciones printf y scanf, para tomar valores de una terminal y comprobar cada funcion que se este ejecutando.

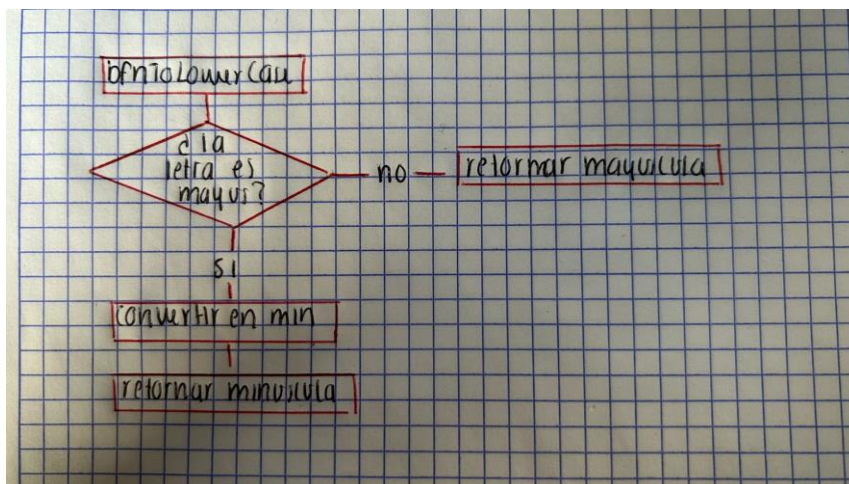
Desarrollo

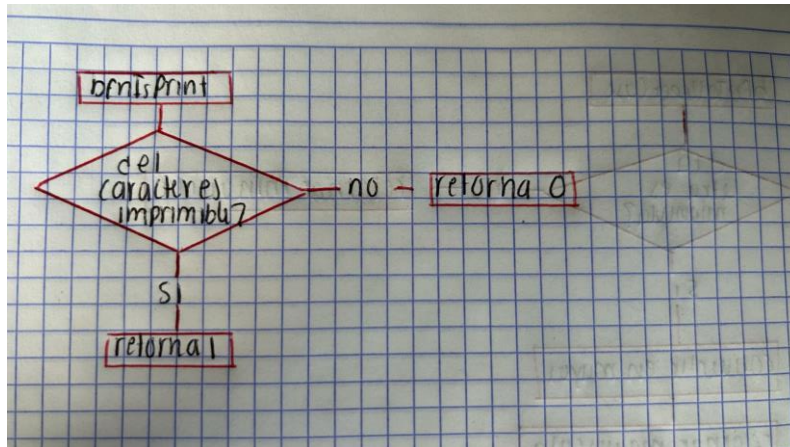
Las funciones por realizar son las siguientes:

1) uint8_t bfnToUpperCase (uint8_t bChar)

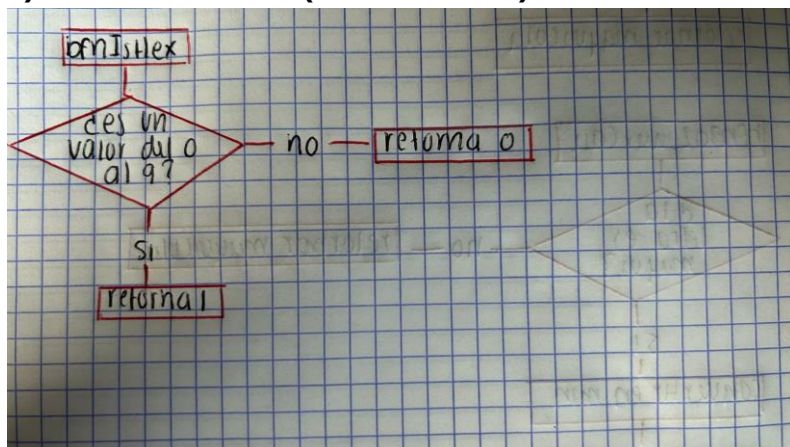


2) uint8_t bfnToLowerCase (uint8_t bChar)

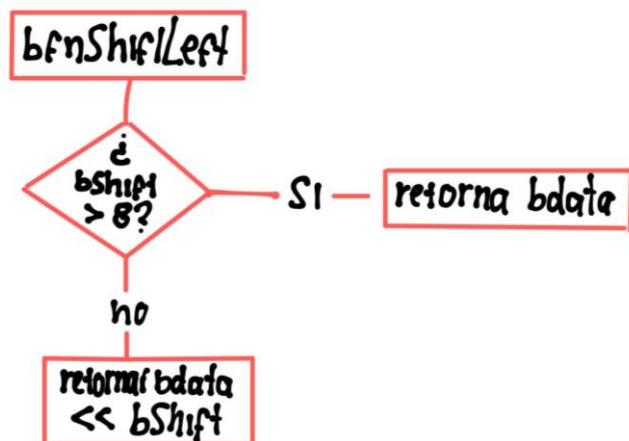




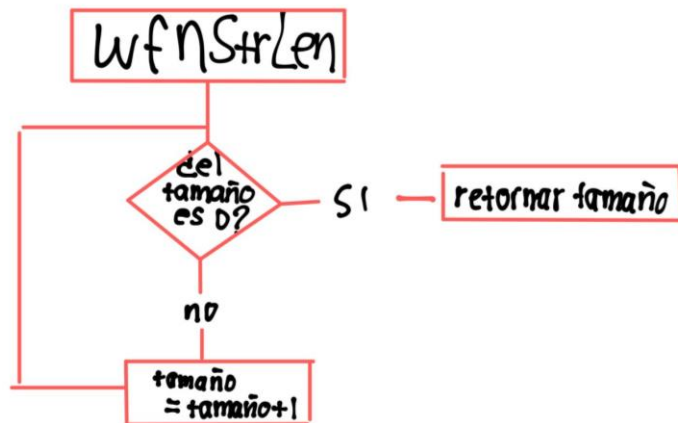
4) `uint8_t bfnIsHex (uint8_t bChar)`



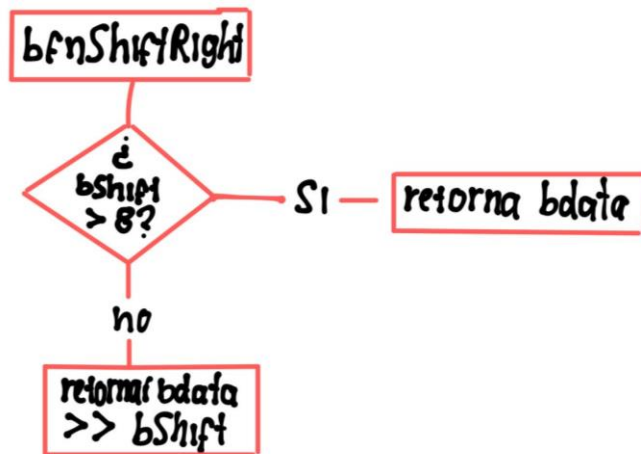
5) `uint16_t wfnStrLen (uint8_t *bpString)`



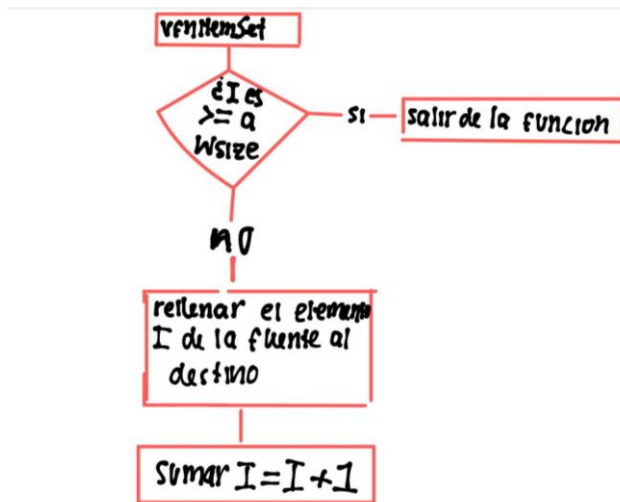
6) `uint8_t bfnShiftLeft (uint8_t bData, uint8_t bShift)`



7) `uint8_t bfnShiftRight (uint8_t bData, uint8_t bShift)`



8) `void vfnMemCpy (uint8_t *bpSource, uint8_t *bpDest, uint16_t wSize)`



The screenshot shows the implementation of `vfnMemSet` in a C file. The function is defined as follows:

```

void vfnMemSet(uint8_t *bpDest, uint8_t bByteToFill, uint16_t wSize)
{
    for (uint16_t i = 0; i < wSize; i++) {
        bpDest[i] = bByteToFill; // Copy each byte from source to destiny
    }
    vfnGreenOn();
}
  
```

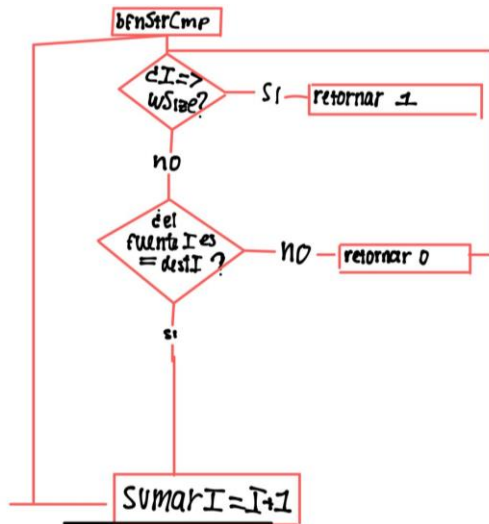
The debugger shows the following registers:

Name	Value	Description
r0	0x20000004	Argument/Scratch Register 1
r1	0x20000024	Argument/Scratch Register 2
r2	0x3	Argument/Scratch Register 3
r3	0x3	Argument/Scratch Register 4
r4	0x180	Variable Register 1
r5	0x14005058	Variable Register 2
r6	0xc00f301	Variable Register 3
r7	0x2003fc0	Variable Register 4
r8	0x5ac3c35a	Variable Register 5
r9	0x33cc33c	Variable Register 6
r10	0x5ac3c35a	Variable Register 7
r11	0x0	Variable Register 8

The memory window shows the following data:

Address	Value
0x20000024	281E140A 00000032 00000000 00000000 ... (2)
0x20000034	00000000 00000000 00000000 00000000
0x20000044	00000000 00000000 00000000 00000000
0x20000054	00000000 00000000 00000000 00000000
0x20000064	00000000 00000000 00000000 00000000
0x20000074	00000000 2000009c 00000000 00000000
0x20000084	00000401 2000009c 00000000 00000000
0x20000094	00000001 00000000 00000000 00000000
0x200000A4	00000000 00000000 00000000 00000000
0x200000B4	200000D8 00000000 00000000 00000402
0x200000C4	200000D8 00000001 00000000 00000001
0x200000D4	00000000 00000000 00000000 00000000
0x200000E4	00000000 00000000 00000000 20000114
0x200000F4	00000000 00000000 00000402 20000114
0x20000104	00000002 00000000 00000001 00000000
0x20000114	00000000 00000000 00000000 00000000

9) void vfnMemSet(uint8_t *bpDest, uint8_t bByteToFill, uint16_t wSize)



The screenshot shows a C code editor with the following functions:

```

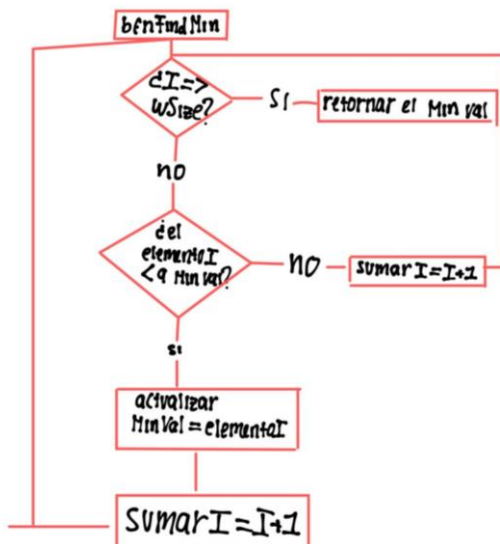
91 void vfnMemcpy(uint8_t *bpSource, uint8_t *bpDest, uint16_t wSize)
92 {
93     for (uint16_t i = 0; i < wSize; i++) {
94         bpDest[i] = bpSource[i]; // Copy each byte from source to destiny
95     }
96     vfnGreenOn();
97 }
98
99 void vfnMemSet(uint8_t *bpDest, uint8_t bByteToFill, uint16_t wSize)
100 {
101     for (uint16_t i = 0; i < wSize; i++) {
102         bpDest[i] = bByteToFill; // fills each byte with the destiny
103     }
104     vfnGreenOn();
105 }
106
107 uint8_t bfnFindMax(uint8_t *bpDest, uint16_t wSize)
108 {
109     uint8_t maxVal = 0;
110     for (uint16_t i = 0; i < wSize; i++) {
111         // ...
112     }
113 }
  
```

The debugger window shows the registers for the LPC55569 (cortex-m3). The register table is as follows:

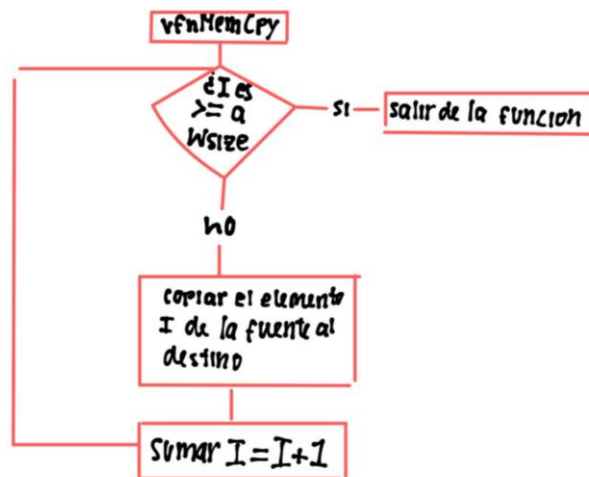
Name	Value	Description
r0	0x20000024	Argument/Scratch Register 1
r1	0x0	Argument/Scratch Register 2
r2	0x3	Argument/Scratch Register 3
r3	0x5	Argument/Scratch Register 4
r4	0x180	Variable Register 1
r5	0x14005058	Variable Register 2
r6	0xc09f301	Variable Register 3
r7	0x2003ffc8	Variable Register 4
r8	0x5ac3c35a	Variable Register 5
r9	0xc3cc33c	Variable Register 6
r10	0x5ac3c35a	Variable Register 7
r11	0x0	Variable Register 8

The memory window shows the address range 0x20000024 to 0x20000024, with a value of 0x20000024.

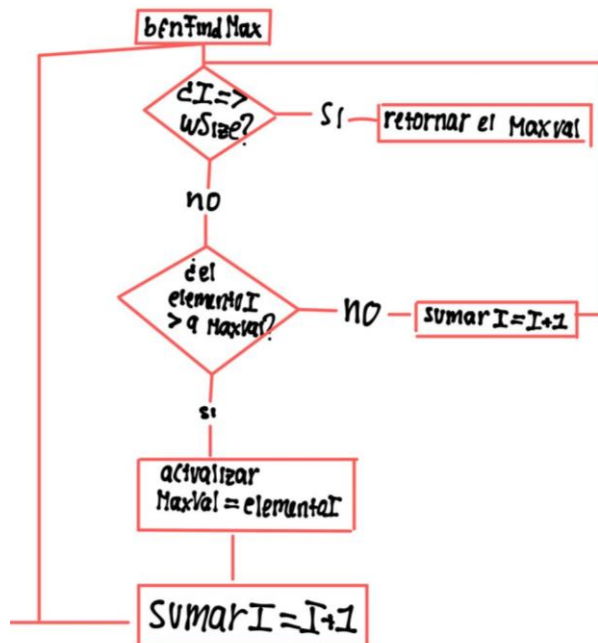
10) `uint8_t bfnFindMax(uint8_t *bpDest, uint16_t wSize)`



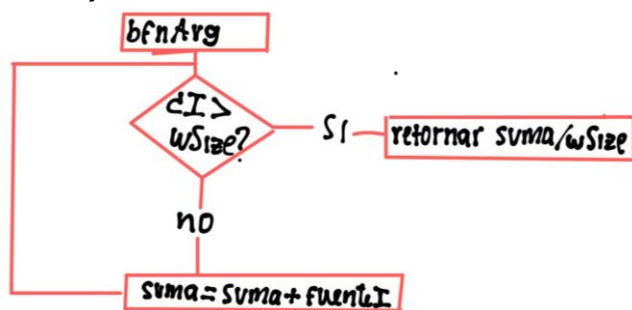
11) `uint8_t bfnFindMin(uint8_t *bpDest, uint16_t wSize)`



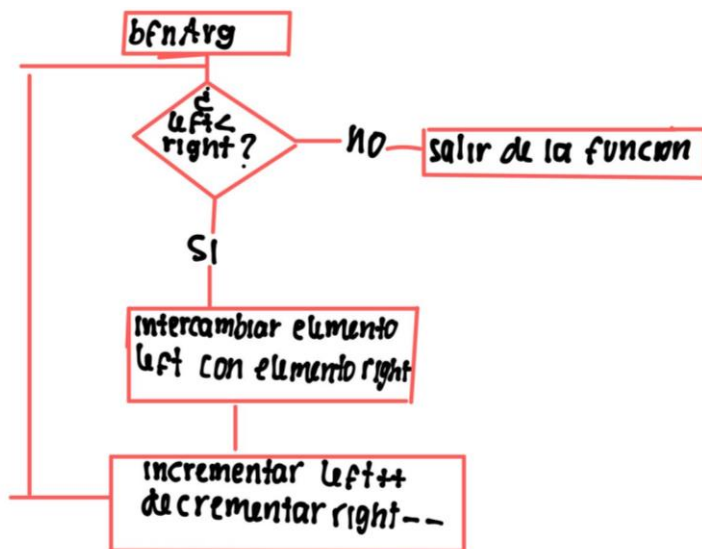
12) `uint8_t bfnStrCmp(uint8_t *bpSource, uint8_t *bpDest, uint16_t wSize)`



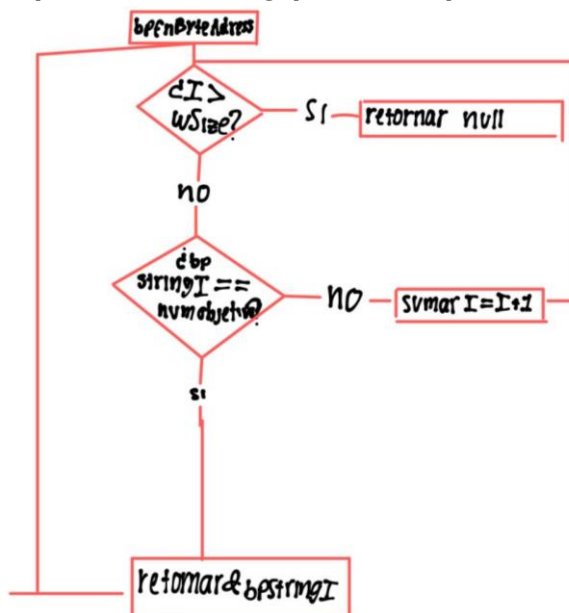
13) `uint8_t *bpfnByteAddress(uint8_t *bpString, uint8_t bCharToFind, uint16_t wSize)`



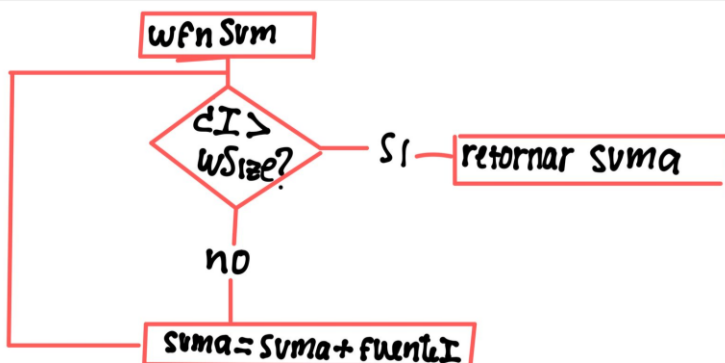
14) `uint16_t wfnSum (uint8_t *bpSrc, uint8_t bSize)`



15) `uint8_t bfnAvg (uint8_t *bpSrc, uint8_t bN)`



16) Inventa o investiga una función que pudiera ayudar a repasar el uso de apuntadores.



Name	Type	Value
bUpperCase	uint8_t	65 'A'
bLowerCase	uint8_t	97 'a'
bResult	uint8_t	0 '\0'
length	uint16_t	7
shiftedLeft	uint8_t	10 '\n'
shiftedRight	uint8_t	2 '\002'
maxVal	uint8_t	50 '2'
minVal	uint8_t	10 '\n'
byteAddr	uint8_t *	0x20000006 <testArray+2>...
sum	uint16_t	150
avg	uint8_t	30 '\036'

Imagen que muestra como terminan luciendo todas las variables utilizadas en el programa.

```

COM3 - Tera Term VT
File Edit Setup Control Window Help
The letter before the function bfnToUpperCase: a
The letter after the function: A
The letter before the function bfnToLowerCase: A
The letter after the function: a
Is the letter printable? : 1 = Yes, 0 = No
The result is 1
Is the character a hexadecimal value? : 1 = Yes, 0 = No
The result is 0
String length: 7
Shift Left result: 10
Shift Right result: 2
Memory Copy completed.
Memory Set completed.
Max Value: 50
Min Value: 10
String Compare result: 0
Byte Address: 20000006
Sum: 150
Average: 30
Array reversed.

```

Así luce la terminal después de que todas las funciones fueron ejecutadas

Comentarios:

Katya

Al inicio no sabíamos cómo empezar la práctica, pero vimos el video del profesor y utilizamos su archivo ya hecho de la inicialización de los leds del microcontrolador (aprovechando que los usaríamos para la práctica), después de hacer las primeras funciones entendimos más como hacer las demás.

Gabriel

Desarrollar esta practica fue un reto por que fueron muchas funciones, la mayoría fueron rápidas y con la ayuda de mi equipo las pudimos terminar a

tiempo, las que mas costaron fueron las que utilizaban apuntadores de dirección porque todavía no nos terminamos de acostumbrar a la sintaxis y su uso en C.